From a 2010 Prototype to Contraction Hierarchies in Three Languages

Engineering k Shortest Paths with Eppstein, Yen–Lawler, Arc Flags and PHAST
An experiment in algorithm engineering with Claude

Nick Dinges*

November 2026

Abstract

We report on turning a small C# implementation of Eppstein's k shortest paths algorithm, written in 2010, into a library that combines several known techniques: Eppstein's enumeration with lazily built persistent sidetrack heaps, a detour bound that limits both the shortest-path tree and the enumeration, Yen's algorithm with Lawler's improvement and A* spur searches for loopless paths, arc flags with external graph partitions, and a contraction hierarchy whose PHAST sweep computes the full shortest-path tree. The library exists in three languages with the same interface and results: C# and Go, which are garbage collected, and Rust, which is not. All three share golden test data and an identical benchmark, so their timings can be compared on exactly the same graphs. On synthetic grid graphs with up to one million vertices, the steady-state time of the shortest-path tree differs by at most a factor of 1.6 between the three implementations for queries of at least a millisecond, and no language is fastest everywhere. Measured naively, on the second query of a process, the gap is up to 5.2×; this is .NET's tiered JIT compilation, not garbage collection. Garbage collection itself is a small cost in time: enumerating 10^5 paths allocates about 840 MB in each language, and the collectors pause for 0.46 ms (Go) and 120 ms (C#) of about 3 s. Its price is memory: the garbage-collected processes need 1.6–1.8× the peak memory of the Rust process. The algorithmic choices dominate by far: the original 2010 implementation needs 40.3 s for the first path on a 12×12 acyclic grid where the current one needs 0.12 ms, and a detour bound turns a 75 ms query into 0.52 ms. Finally, we describe how the work was done: the whole transformation, including this paper, was generated with Claude in about three days, directed by the author and based on the author's old code and on ideas from a domain expert. We report what worked, what the AI got wrong, and how tests and a critical review of this paper caught it.

1 Introduction

The k shortest paths problem asks for the k shortest paths between two vertices s and t of a weighted directed graph, in order of weight. It appears in route planning (alternatives to the best route), network routing, and as a building block of optimization and language-processing pipelines. There are two main variants: paths may repeat vertices (*walks*), or they must be *simple* (loopless). For the first variant, Eppstein [17] gave an algorithm that, after one shortest-path tree computation, produces the k shortest paths in order in $O(m + n \log n + k \log k)$ total time, in an implicit representation; it is still the reference today. For simple paths, Yen's algorithm [36] with Lawler's improvement [31] is the classic choice, with $O(kn(m + n \log n))$ time in the worst case; sidetrack-based algorithms are a recent improvement in practice [29, 37].

*This paper is the result of an experiment: the code in all three languages, the tests, the measurements and the text were generated with Claude, Anthropic's AI model, through Claude Code [3], under the author's direction. See section 7.

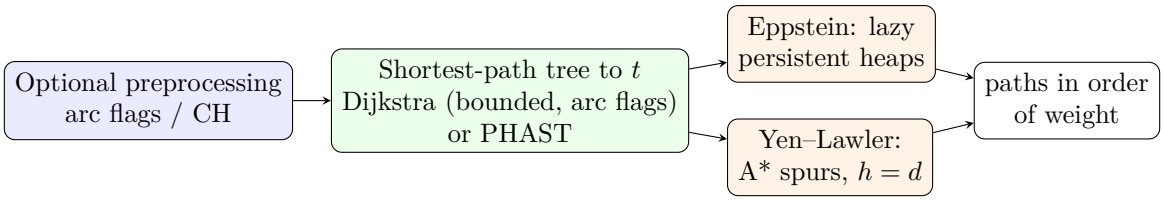


Figure 1: A query. The tree supplies the distances $d(v)$ to the goal, which Eppstein uses for the sidetrack deltas and Yen–Lawler as exact A* lower bounds. The detour bound limits the tree, the enumeration and the spur searches.

This paper is an engineering report, not a new algorithm. Its starting point is a 486-line C# implementation of Eppstein's idea written by the author in 2010. It computed correct paths on the example of Eppstein's paper, but it enumerated all paths eagerly in the first query, its priority queue was a sorted list, and it did not terminate on graphs with cycles. In November 2026 the author ran an experiment: could Claude, an AI model used as a coding agent, turn it into a modern library? The project grew in small steps, each one requested by the author: fix the non-termination, port to Linux, add Graphviz output, port to Go, make it fast, add simple paths, add arc flags with real partitions, add contraction hierarchies, port to Rust, write this paper, and finally review the paper critically.

Contributions.

- An exact combination of known techniques for k shortest paths in one library (section 2): lazy Eppstein enumeration with persistent leftist heaps, a detour bound that limits the tree *and* the enumeration, Yen–Lawler with A* spur searches guided by the exact distances of the shortest-path tree, arc flags with partitions from external partitioners, and a contraction hierarchy with PHAST for the tree. None of the techniques is new; we explain each of them from the definitions onward, describe what has to be adapted to combine them (the tree must be rebuilt after PHAST; arc flags cannot skip vertices), and test every combination against brute force.
- Three implementations with identical interfaces and results in C#, Go and Rust, sharing golden test data, and an identical benchmark (same pseudo-random graphs, same output). This allows a comparison of two garbage-collected languages and one without a collector on the same algorithms and nearly the same data layout, including direct measurements of allocation, collections and JIT warm-up (sections 4 to 6).
- A case study of AI-assisted algorithm engineering (section 7): which inputs came from the human (an old algorithm, short requests, expert ideas), what Claude did, and where it failed, in the code and in an earlier version of this paper.

2 The algorithms, step by step

This section explains every algorithm the library uses, from the definitions onward, so that the paper can be read without the original articles. Readers who know Eppstein's algorithm, Yen's algorithm and contraction hierarchies can skip to the paragraphs marked *In the library*, which say what our implementation does differently. Figure 1 shows how the parts work together in a query.

2.1 Definitions

A directed graph $G = (V, E)$ has n vertices and m edges. An edge $e = (u, v)$ leads from its *tail* u to its *head* v and has a weight $w(e) \geq 0$; several edges may connect the same two vertices. A *walk* from s to t is a sequence of edges in which each edge starts where the previous one ends; its weight is the sum of its edge weights. A walk may visit a vertex several times. A walk that visits no vertex twice is a *simple path*. We say "path" for both where the difference does not matter. The *distance* $d(u, v)$ is the smallest weight of a walk from u to v, and infinite if there is none. Given s, t and k, the k *shortest paths problem* asks for k walks from s to t of smallest weight, in order of weight; the k *shortest simple paths problem* asks the same for simple paths. If a graph has a cycle that can be reached from s and from which t can be reached, it has infinitely many walks from s to t but only finitely many simple paths.

Notation for running times. We state how the cost of each algorithm grows with the input, in the usual O-notation: a running time is $O(f)$ if it is at most a constant times f for all large enough inputs. Constant factors and lower-order terms are ignored, so the notation describes scaling, not speed on a particular machine; the experiments in section 5 complement it. Throughout, n is the number of vertices, m the number of edges (we assume that m is at most polynomial in n, so $\log m = O(\log n)$), k the number of requested paths, $\deg(v)$ the number of edges leaving v, and ℓ the largest number of edges of a path that is output. All bounds are for the worst case unless we say otherwise. Table 1 at the end of this section collects them.

2.2 Dijkstra's algorithm and the shortest-path tree

All algorithms below start from the distances of all vertices *to the goal*: $d(v) = d(v, t)$. Dijkstra's algorithm [14] computes them. It keeps a tentative distance for every vertex, initially 0 for t and infinite for all others, and a priority queue of vertices ordered by tentative distance. It repeatedly removes the vertex v with the smallest tentative distance, which is then final (*settled*), and *relaxes* every edge (u, v) entering v: if $w(u, v) + d(v)$ is smaller than the tentative distance of u, it becomes the new one, and (u, v) is remembered as the *tree edge* of u. Because weights are not negative, a settled distance can never be improved later. Since the search starts at t and follows edges backwards, we call it a *backward* search. Every vertex is removed from the queue once and every edge is relaxed once. With a binary heap as priority queue, each queue operation costs $O(\log n)$, so the algorithm takes $O((n + m) \log n)$ time and $O(n + m)$ space; this is what the library uses. With a Fibonacci heap the bound improves to $O(m + n \log n)$, which is the figure usually quoted in the literature, but binary heaps are faster in practice.

The tree edges form the *shortest-path tree* T: every vertex that can reach t has exactly one tree edge, to a vertex we call $\text{next}(v)$, and following tree edges from any vertex leads to t along a shortest path. An edge (u, v) is *tight* if $d(u) = w(u, v) + d(v)$, i. e. if it lies on some shortest path to t; tree edges are tight, and for every edge $d(u) \leq w(u, v) + d(v)$.

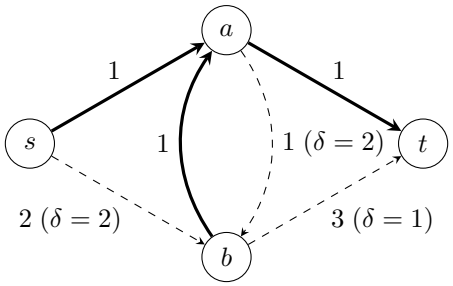
2.3 Eppstein's algorithm: paths as sequences of sidetracks

Sidetracks. Fix the tree T to the goal t. An edge $e = (u, v)$ that is not a tree edge is called a *sidetrack*. Its *delta*

$$\delta(e) = w(e) + d(v) - d(u) \ \geq 0$$

is the extra weight of leaving u over e and continuing optimally, compared with the shortest path from u. Take any walk from s to t and add up the deltas of all its edges (tree edges have delta 0): the distances cancel in pairs, and the sum is the weight of the walk minus $d(s)$. Hence

$$\text{weight of a walk} = d(s) + \sum_{\text{its sidetracks } e} \delta(e).$$



vertex	$d(v)$	$H(v)$, smallest first
t	0	empty
a	1	(a, b)
b	2	(b, t), (a, b)
s	2	(a, b), (s, b)

	sidetrack sequence	walk	weight
1	(none)	$s\,a\,t$	2
2	(a, b)	$s\,a\,b\,a\,t$	$2 + 2 = 4$
3	(s, b)	$s\,b\,a\,t$	$2 + 2 = 4$
4	$(s, b), (b, t)$	$s\,b\,t$	$2 + 2 + 1 = 5$
5	$(a, b), (b, t)$	$s\,a\,b\,t$	$2 + 2 + 1 = 5$
6	$(s, b), (a, b)$	$s\,b\,a\,b\,a\,t$	$2 + 2 + 2 = 6$

Figure 2: Example. Thick edges: the shortest-path tree to t; dashed edges: sidetracks with their deltas. $H(v)$ contains the sidetracks that leave the tree path from v to t. Below: the six shortest walks and the sidetracks that describe them. The graph has a cycle ($a \to b \to a$), so there are infinitely many walks.

A walk is also determined by its sequence of sidetracks alone: between two sidetracks, and after the last one, it has no choice but to follow tree edges. In fig. 2, the sidetrack sequence $(s, b), (b, t)$ means: take (s, b), then, standing at b, immediately take (b, t); its weight is $d(s) + 2 + 1 = 5$. Not every sequence is valid: after a sidetrack with head v, the next sidetrack must start at a vertex on the tree path from v to t. So the problem becomes: *enumerate the valid sidetrack sequences in order of their sum of deltas.* The empty sequence is the shortest path.

The heaps $H(v)$. For a vertex v, let $H(v)$ be the set of sidetracks whose tail lies on the tree path from v to t; these are exactly the sidetracks that may follow a sidetrack with head v (and $H(s)$ contains the possible first sidetracks). To find the best ones quickly, $H(v)$ is stored as a *heap*: a binary tree whose nodes hold one sidetrack each, such that the delta of a node is never larger than the deltas of its children. The smallest element is at the root, and everything below a node is at least as large as the node.

Storing each $H(v)$ separately would need far too much space, since $H(v)$ consists of the sidetracks leaving v itself plus all of $H(\text{next}(v))$. Eppstein's key idea is to let the heaps share nodes: $H(v)$ is built from $H(\text{next}(v))$ by adding the sidetracks of v *without changing* $H(\text{next}(v))$, which other vertices still use. A data structure whose old versions stay valid after an update is called *persistent* [16]. Here it is obtained by copying the few nodes an update would change instead of overwriting them.

Enumeration. A *candidate* is a sidetrack sequence. The algorithm keeps a priority queue of candidates ordered by weight, starting with the empty sequence, and repeats: remove the lightest candidate, output it as the next path, and insert its *successors*. For a candidate whose last sidetrack e was taken from a node x of the heap $H(v)$, the successors are

- *replace:* the same sequence with e replaced by the sidetrack of a child of x in the heap (at most two successors), and
- *append:* the sequence extended by the root of $H(\text{head}(e))$, the best sidetrack that can follow e (at most one successor).

The empty sequence has one successor, the root of $H(s)$. Two facts make this correct. A successor is never lighter than its candidate, because a heap child is not smaller than its parent and deltas are not negative; so when a candidate is removed from the queue, no lighter one can appear later. And every valid sequence is generated exactly once: its last sidetrack is one node of the heap that belongs to the sequence before it, and there is exactly one way down from the root of that heap to this node. In the example, the candidate (a, b), the root of $H(s)$, produces (s, b) by replacement and $(a, b), (b, t)$ by appending the root of $H(b)$, the heap at the head of (a, b). Each output costs one removal and at most three insertions, so after k outputs the queue holds at most $3k$ candidates, and producing the k lightest sequences takes $O(k \log k)$ time and $O(k)$ space once the tree and the heaps exist. The graph whose vertices are the heap nodes and whose edges are these successor relations is Eppstein's *path graph*. With all heaps built in advance in a refined way, his algorithm takes $O(m + n \log n + k \log k)$ time in total [17]: $O(m + n \log n)$ for the tree and the heaps, $O(k \log k)$ for the enumeration in order. This counts each path as its sidetrack sequence (an *implicit* representation); writing a path out as a list of edges costs additional time proportional to its length, $O(k\ell)$ in total.

In the library: lazy heaps with leftist merging. Building the heaps of all vertices in advance is wasteful when only a few paths are requested. Jiménez and Marzal [27] build $H(v)$ only when the enumeration first needs it, and we follow them: $H(v)$ is requested for $v = s$ and for the heads of sidetracks of extracted candidates; if $H(\text{next}(v))$ does not exist yet, it is built first. The library does this iteratively, from the first vertex on the tree path that already has a heap back towards v, so long tree paths cause no deep recursion. The first candidate therefore needs the heaps of all vertices on the tree path from s to t.

For the heaps we use *leftist heaps* [9]. The *rank* of a node is the number of nodes on the path that always goes to the right child, down to a missing child. A heap is leftist if at every node the rank of the left child is at least the rank of the right child; then the rightmost path of a heap with N nodes has at most $\log_2(N + 1)$ nodes. Two leftist heaps are *merged* by comparing the roots, keeping the smaller one as the new root, merging its right subtree with the other heap in the same way, and finally swapping the two children if the right one has the larger rank. Only nodes on the two rightmost paths are touched. For persistence, the merge writes each touched node as a new copy and leaves the old one unchanged, so both input heaps remain valid and the result shares all untouched nodes with them. Eppstein keeps the sidetracks of each vertex in a small heap of their own and inserts only its root into a balanced heap along the tree path. We simplify this: the sidetracks leaving v are sorted by delta and linked into a chain through their left children (a valid leftist heap whose rightmost path has length one), and this chain is merged into $H(\text{next}(v))$. The merge copies $O(\log m)$ nodes, so $H(v)$ costs $O(\deg(v) \log \deg(v) + \log m)$ time and, beyond one node per sidetrack, $O(\log m)$ additional nodes.

Complexity of our variant. Even if the enumeration eventually needs the heap of every vertex, all heaps together cost $O(m \log n)$ time for sorting and $O(n \log n)$ for merging, and $O(m + n \log n)$ space. With the binary heap in Dijkstra's algorithm the whole query is therefore

$$O\big((n + m) \log n + k \log k\big) \text{ time and } O(m + n \log n + k) \text{ space}$$

for k paths in implicit form, plus $O(k\ell)$ time to write them out. This is a factor of at most $\log n$ above Eppstein's bound in the first term and equal in the second. Laziness does not improve the worst case but the typical one: the first path needs only the heaps of the L vertices on the tree path from s to t, at a cost of $O(L \log n)$ plus the sorting of their sidetracks, and each further path needs at most the heaps on one further piece of tree path.

A candidate is stored as a pointer to its parent candidate and one heap node, so creating one costs $O(1)$, and its weight is the weight of the sequence it extends plus one delta. To output a path as a list of edges, the library collects the sidetracks by following the parent pointers and then walks from s to t, taking the next sidetrack where its tail is reached and tree edges

otherwise. This costs time proportional to the length of the path and dominates in practice (table 4); an iterator that returns only the weights skips it.

2.4 Detour bound ("corridor")

Users rarely want arbitrarily long alternatives. With an absolute or relative detour limit Δ, only paths of weight at most $d(s) + \Delta$ are admissible; enumerating them is known as the near-shortest paths problem [6, 7]. The bound prunes all three stages of a query. A vertex v with $d(v) > d(s) + \Delta$ cannot lie on an admissible path, so the backward Dijkstra stops as soon as it would settle a vertex beyond the radius $d(s) + \Delta$ (the radius becomes known when s is settled, which happens before any vertex farther away than s is settled). In the enumeration, a candidate with $\sum \delta > \Delta$ is not inserted into the queue, and since successors are never lighter, nothing is lost. The A* spur searches of section 2.5 stop in the same way. The library calls this the *corridor*. Two remarks put it into perspective. First, the region that is searched is a ball around t, not an ellipse around the s–t path: the exact condition for a vertex is $d(s, v) + d(v) \leq d(s) + \Delta$, which needs forward distances from s as well. A bidirectional search, or K* [2], which builds the path graph while an A* search explores the graph, would search a smaller region; our bound is simply the part that a single backward search gives for free. It helps when s and t are close relative to the size of the graph, and not at all when they are far apart. Second, with the bound the number of admissible paths is finite even with cycles, provided that no cycle of weight zero lies on an admissible walk; a query can then be iterated to its end.

Complexity. The bound does not change the worst case, since the ball may be the whole graph. It replaces n and m in all bounds by the numbers n_Δ and m_Δ of vertices and edges within distance $d(s) + \Delta$ of t: the tree costs $O((n_\Delta + m_\Delta) \log n_\Delta)$. For a query between nearby vertices this makes the cost independent of the size of the graph, which is the effect measured in table 5.

2.5 Simple paths: Yen's algorithm with Lawler's rule and A*

In networks with two-way edges most of the shortest walks are the shortest path plus a small loop (in fig. 2, walks 2 and 6), not real alternatives. With `SimplePathsOnly`, the library returns simple paths only, using a different algorithm, since no way is known to restrict Eppstein's path graph to simple paths without checking each walk.

Yen's algorithm. The first path P_1 is the shortest path, which is simple. Every other simple path Q from s to t *deviates* from each previously found path somewhere. Yen's algorithm [36] uses this to derive new paths from found ones. Let $P = (v_0 = s, v_1, \ldots, v_\ell = t)$ be a path that has just been output. For each position $i < \ell$, the prefix $(v_0, \ldots, v_i)$ is called the *root* and v_i the *spur vertex*. The algorithm computes the shortest path from v_i to t, the *spur*, in a restricted graph in which

- the vertices $v_0, \ldots, v_{i-1}$ of the root are removed, so that root plus spur is simple, and
- every edge leaving v_i that some already found path with the same root uses next is removed, so that the result differs from all found paths.

Root plus spur is the best simple path that shares exactly this root with P and then leaves it, and it becomes a *candidate*. The next path to output is the lightest candidate found so far, kept in a priority queue. The algorithm is correct because the next simple path, whatever it is, shares some root with a found path and then leaves all found paths with that root, so it is the best path of one of the restricted problems that have been solved. Each output path has at most $\ell \leq n - 1$ edges and therefore causes at most ℓ shortest-path computations, which gives $O(k\ell(m + n \log n))$, or $O(kn(m + n \log n))$ in terms of the graph alone, with Fibonacci heaps. No algorithm with a substantially better worst-case bound is known for directed graphs. Compared with walks, the

cost per path grows from $O(\log k)$ to about ℓ shortest-path computations, which is the price of simplicity.

Lawler's rule. Every path except P_1 was created as a candidate from a parent path at some position j, its *deviation index*: up to v_j it equals its parent. Lawler [31] observed that for positions $i < j$ the restricted problem is the same one that was already solved when the parent was processed: the root is the same, and the new path adds no removed edge there, because at v_i it uses the same edge as its parent. Only positions $i \geq j$ need spur computations. This removes duplicate candidates and many searches, but a path may still deviate at its first vertex, so the worst-case bound is unchanged.

A* with exact distances. A spur is a point-to-point shortest path, for which Dijkstra's algorithm explores much more of the graph than necessary. A* search [22] runs forward from the spur vertex and orders its priority queue by $g(v) + h(v)$, where $g(v)$ is the tentative distance from the spur vertex and $h(v)$ a lower bound (*potential*) on the remaining distance to t. If $h(u) \leq w(u, v) + h(v)$ holds for all edges (*consistency*), every vertex is settled at most once and the search may stop when t is settled. The better the bound, the fewer vertices are explored. Here an excellent bound is available for free: the distances $d(v)$ of the shortest-path tree of the unrestricted graph. Removing vertices and edges can only make distances longer, so d is a lower bound in every restricted graph, and it is consistent (section 2.2). When the tree path from the spur vertex survives the restrictions, the search settles only the vertices of that path; otherwise it explores only where the restrictions force a detour. In the worst case an A* search is no better than Dijkstra's algorithm, $O((n + m) \log n)$ with a binary heap. In the best case, a spur of ℓ' edges whose tree path is free costs $O(\ell' \, \Delta_{\max} \log n)$, where $\Delta_{\max}$ is the largest number of edges leaving a vertex: only the vertices of the spur are settled, and only their edges are relaxed.

In the library. The found paths are stored in a trie (a tree in which each path from the root spells the edge sequence of a prefix); the edges to remove at a spur vertex are the children of the root's trie node. The tree to t is computed once per query, as for Eppstein. Spurs of a path are generated lazily when the next path is requested, so asking for one path costs no spur search. With a detour bound, a spur search stops as soon as the smallest $g + h$ in its queue exceeds what an admissible path may still cost. With binary heaps the worst-case time of a query is $O(k\ell(n + m) \log n)$. The space is $O(n + m)$ for the graph and the search plus the candidates: up to ℓ per output path, each stored as a full edge list, so $O(k\ell^2)$ in the worst case; this, not time, can become the limit for long paths and large k. Using the reverse tree to guide or avoid spur searches is known [18, 29]; we do not yet skip the search when the tree path itself is a valid spur, which those algorithms do.

2.6 Arc flags

The remaining techniques invest preprocessing time to make the shortest-path tree of later queries cheaper. For *arc flags* [30, 32], the vertices are partitioned into a small number of *regions*. Every edge gets one bit per region; the bit of edge e for region R is set if e lies on a shortest path to some vertex of R. A search for a goal in region R may then ignore all edges whose bit for R is not set, because no shortest path to the goal uses them.

The bits are computed as follows. An edge inside R gets the bit for R. A shortest path that enters R from outside must pass through a *boundary vertex* of R, a vertex of R with an incoming edge from another region, and up to that vertex it is a shortest path to the boundary vertex. So for every boundary vertex b of R, one backward Dijkstra from b computes the distances to b, and every edge that is tight for these distances gets the bit for R. With r regions and b boundary vertices in total, the preprocessing costs one Dijkstra run per boundary vertex, $O(b\,(n + m) \log n)$

time, which is why partitions with few boundary vertices matter, and the flags need $O(mr)$ bits. In a grid-like graph cut into r regions, b grows like $\sqrt{rn}$, so the preprocessing is clearly superlinear; it pays off only over many queries.

In the library. The partition comes from chunks of a breadth-first order (built in), from a string, or from an external partitioner such as Mt-KaHyPar [33], for which the library writes the METIS input format. The backward Dijkstra of a query relaxes only edges flagged for the goal's region. Every vertex still gets its correct distance, because its shortest path to t consists of flagged edges. But since Eppstein needs the distance of *every* vertex that can reach the goal, arc flags cannot skip vertices here, as they do in a point-to-point query; they only skip edge relaxations. The query remains $O((n + m_R) \log n)$, where $m_R \leq m$ is the number of edges flagged for the goal's region: the term for the vertices is untouched, so the gain is small by construction (table 8). The documentation recommends the detour bound first.

2.7 Contraction hierarchy and PHAST

Contraction. A *contraction hierarchy* (CH) [19] removes the vertices from the graph one by one. To *contract* a vertex v means to remove it while preserving all distances between the remaining vertices: for every remaining in-neighbor u and out-neighbor x of v, if the path $u \to v \to x$ is the only shortest path from u to x, a *shortcut* edge (u, x) with weight $w(u, v) + w(v, x)$ is added. Whether another path of at most that weight exists is decided by a *witness search*, a Dijkstra search from u that avoids v. It may be cut off after a fixed number of settled vertices; if it then has found no witness, the shortcut is added anyway, which costs space but never correctness. The order of contraction defines the *rank* of each vertex. A good order contracts unimportant vertices first and keeps the number of shortcuts small; the usual heuristic picks the vertex with the smallest *edge difference* (shortcuts added minus edges removed) next.

Contraction hierarchies are a heuristic in the sense of running time: the results are exact, but no useful worst-case bound holds for general graphs. Contracting a vertex with a incoming and c outgoing edges runs a witness searches and adds up to ac shortcuts, and in dense or hierarchy-free graphs the number of shortcuts can grow to $O(n^2)$. On road networks, which have a natural hierarchy of roads, the number of shortcuts stays close to m in practice [19]. We write m^+ for the number of edges of the graph with shortcuts.

Let G^+ be the original graph plus all shortcuts. Call an edge *upward* if it leads to a vertex of higher rank and *downward* otherwise. The construction guarantees that for any two vertices u and t with a path between them, G^+ contains a shortest path from u to t that first uses only upward edges and then only downward edges (an *up–down path*): wherever a shortest path has a vertex that is lower than both its neighbors on the path, that vertex was contracted before them, and either a shortcut or a witness path replaces the dip. A point-to-point query exploits this with two small searches, one upward from each end.

PHAST. Eppstein needs the distances of all vertices to t, not one. PHAST [10] computes them from the hierarchy in two phases.

1. *Upward search from t.* A backward Dijkstra from t that follows only downward edges, backwards. It reaches only vertices of higher rank than t, typically a few hundred, and gives each of them the length of the best purely downward path to t.

2. *Sweep.* All vertices are processed once, in order of descending rank. For each vertex u, every upward edge (u, x) is relaxed: $d(u) \leftarrow \min(d(u), w(u, x) + d(x))$.

After the sweep every $d(u)$ is correct, by induction over the rank: the shortest up–down path from u is either purely downward, and was found in phase 1, or it starts with an upward edge (u, x) and continues with a shortest up–down path from x, whose distance is already final because

Table 1: Worst-case bounds of the algorithms as implemented (binary heaps). n vertices, m edges, k paths, ℓ edges in the longest output path, r regions, b boundary vertices, m_R edges flagged for the goal's region, m^+ edges including shortcuts, q size of the upward search space, n_Δ, m_Δ vertices and edges within the detour bound. Writing out the paths adds $O(k\ell)$ time everywhere.

step	preprocessing	query time	space
Tree: Dijkstra	–	$O((n+m)\log n)$	$O(n+m)$
with detour bound	–	$O((n_\Delta+m_\Delta)\log n_\Delta)$	$O(n+m)$
with arc flags	$O(b\,(n+m)\log n)$	$O((n+m_R)\log n)$	$O(n+mr)$
with CH and PHAST	no useful bound	$O(q\log q+n+m+m^+)$	$O(n+m^+)$
Walks: Eppstein (original)	–	$O(m+n\log n+k\log k)$	$O(m+n\log n+k)$
Walks: lazy heaps (ours)	–	$O((n+m)\log n+k\log k)$	$O(m+n\log n+k)$
first path, after the tree	–	$O(L\log n+\sum_v \deg(v)\log\deg(v))$	
Simple: Yen–Lawler, A*	–	$O(k\ell\,(n+m)\log n)$	$O(n+m+k\ell^2)$
2010 prototype	–	$\geq$ number of all walks	all walks

x has a higher rank and was processed earlier. Phase 1 costs $O(q\log q)$ for a search space of q vertices and edges, and the sweep $O(n+m^+)$: every vertex and every upward edge is looked at exactly once. PHAST thus replaces the $O((n+m)\log n)$ of Dijkstra's algorithm by a bound that is linear in the size of the hierarchy. Asymptotically the gain is only the logarithmic factor, and only if m^+ is not much larger than m. The practical gain is larger, because the sweep needs no priority queue and reads memory in a fixed order. RPHAST [11] restricts the sweep to the vertices that matter for a given set of targets, and customizable contraction hierarchies [13] separate an order that depends only on the graph's structure from a fast update for new weights.

In the library. The contraction order is maintained lazily (a vertex's priority is recomputed when it reaches the front of the queue) by edge difference plus the number of already contracted neighbors, which spreads the contraction evenly; witness searches have a settled-vertex limit, and of parallel edges only the lightest is kept. Vertices are stored in sweep order. Eppstein also needs the tree, not only the distances. Unpacking shortcuts is unnecessary: we rebuild the tree from the *original* edges that are tight, breadth-first from t. Choosing an arbitrary tight edge per vertex would be wrong if the graph has cycles of weight zero, because the chosen edges could close such a cycle; the breadth-first order prevents this. This pass costs $O(n+m)$, so a tree query with the hierarchy takes $O(q\log q+n+m+m^+)$ time; it is part of the PHAST times we report. The hierarchy needs $O(n+m^+)$ space. With a detour bound the bounded Dijkstra usually visits fewer vertices than PHAST and is used instead. For Yen–Lawler the hierarchy accelerates only the one initial tree, not the spur searches. To our knowledge, the combination of Eppstein's enumeration with PHAST has not been described before; a point-to-point CH query would not be enough for it.

2.8 Summary of the bounds

Table 1 collects the bounds. Three observations follow from them without any measurement. First, the tree dominates a query for walks unless k is very large: each further walk costs $O(\log k)$ plus its output, while the tree costs $O((n+m)\log n)$. Everything that makes the tree cheaper, above all the detour bound, which shrinks n and m themselves, pays off directly, and arc flags, which leave the term $n\log n$ untouched, cannot help much. Second, simple paths are in a different class: every path costs up to ℓ searches of the size of a tree computation, so speeding up the one initial tree is irrelevant for them, and the A* potential, which shrinks each search, is the important lever. Third, the 2010 prototype is not a slow implementation of the same algorithm but a different one: it enumerates every walk before returning the first, and the number of walks

grows exponentially with the size of the graph (in an acyclic $g \times g$ grid there are $\binom{2(g-1)}{g-1}$ from corner to corner). In the row for the first path, L is the number of vertices on the tree path from s to t and the sum runs over these vertices.

2.9 Data layout and exactness

Vertices and edges are stored in flat arrays (structure of arrays): tail, head, weight and group per edge, and compressed adjacency rows by direction, rebuilt lazily after a change. Per-vertex query data (distance, tree edge, heap root) live in arrays that are reused across queries: a vertex's entry is valid only if its *stamp* equals the current query number, so a query touches only the vertices it reaches and never resets the arrays. Public `Vertex`, `Edge` and `Path` types are small values that refer to these arrays. Weights are 64-bit integers; an edge with a negative weight is treated as switched off. All techniques keep the results exact: the same paths with the same weights are returned with and without a detour bound (up to the bound), arc flags and the hierarchy; only paths of equal weight may come in a different order.

3 Related work

The previous section introduced the algorithms we build on; here we place them among the alternatives.

Paths that may repeat vertices. Eppstein's algorithm [17] is the reference for walks. Jiménez and Marzal proposed the recursive enumeration algorithm [26], which computes the k-th path to a vertex from the paths to its predecessors, and later the lazy version of Eppstein's algorithm [27] that we use; both are faster in practice than the original. K* [2] interleaves an A* search with the construction of the path graph, so that the full tree is never needed.

Simple paths. Beyond Yen [36] and Lawler [31], much work reduces the cost of the spur searches: Hershberger et al. [23] use replacement paths, Feng [18] uses the reverse shortest-path tree to restrict each search to the vertices that can matter, and Kurz and Mutzel [29] and Al Zoobi et al. [37] combine Eppstein's sidetrack idea with simplicity checks and avoid many spur searches altogether. Alternative routes [1] and k shortest paths with limited overlap [8] address a related problem, where paths must be sufficiently different.

Speed-up techniques. Arc flags and contraction hierarchies are two of many techniques for route planning in road networks; Bast et al. [5] give a survey.

Languages, garbage collection and measurement. Hertz and Berger [24] quantified that garbage collection can match explicit memory management in speed if it is given several times more memory. Comparisons of the same program in several languages [34, 25] show that the differences between implementations within one language are often as large as those between languages, which is a caveat for our single implementation per language. For runtimes with JIT compilers, start-up and steady state must be separated, and steady state is not always reached [20, 4]; an earlier version of our own measurements illustrates the problem (section 6).

4 Three implementations

Table 2 summarizes the implementations. They follow the same structure file by file; the interfaces differ only where the languages differ (exceptions vs. error values vs. `Result`, properties vs. methods, `IEnumerable` vs. Go iterators vs. Rust iterators).

Table 2: Implementations. Lines exclude blank and comment lines.

	library	tests	notable language-specific choices
2010 original (C#)	486	–	object graph, sorted-list priority queue
C# (.NET 8/10)	1825	1212	`readonly struct` views, `PriorityQueue`, FsCheck
Go 1.23+	1851	1733	generic heap, `iter.Seq`, native fuzzing
Rust 2021	1988	352	arenas with `u32` links, borrow-checked iterators

Memory management. The graph and the per-vertex query data are flat arrays in all three languages. The enumeration differs: in C# and Go, heap nodes and candidates are small objects managed by the garbage collector; in Rust they live in two vectors (arenas) per query and refer to each other by `u32` indices, and the arenas are cleared at the next query. There is no reference counting and no unsafe code in the Rust library. Every returned path is a freshly allocated list of edges in all three.

Safety of iteration. All three offer an iterator over the paths of a query. Changing the graph during the iteration is a runtime error in C# (`InvalidOperationException`) and Go (panic). In Rust the iterator borrows the graph mutably, so the same mistake is a compile-time error; reading the graph during the iteration goes through the iterator.

Labels. Vertex labels are case-insensitive. C# and Go keep the caller's string if it is already in upper case; the Rust version stores all label text in one string and indexes it with an open-addressing hash table, which avoids one heap allocation and one map entry per vertex. (A first, straightforward Rust version stored each label twice and needed 200 MB instead of 112 MB for the largest graph.)

Testing. The three implementations read the same golden files (graphs and expected paths, including DIMACS input), and compare against brute-force enumeration on thousands of random small graphs with cycles, zero and negative weights, detour bounds, arc flags and contraction hierarchies. C# adds property-based tests (FsCheck), Go adds coverage-guided fuzzing of the importers and of the path enumeration. CI enforces coverage gates (C# $\geq 95\%$ lines, Go $\geq 93\%$ statements); the Rust tests are the smallest suite and have no coverage gate.

5 Experiments

Setup. All measurements ran on Intel Xeon Processor @ 2.10GHz (4 virtual cores, Linux) with .NET 10.0.114, Go 1.24.7 and rustc 1.97.0 (release builds; Rust with one codegen unit). The benchmark is the same program in the three languages. It builds a $n \times n$ grid (vertex $v = y\,n + x$; edges to the right and lower neighbor, and every tenth vertex also to its left neighbor, which creates cycles); `two-way` grids have every street in both directions. Weights are $1 + (r \bmod 100)$ for r drawn from SplitMix64 [28] with seed 1, implemented identically in all three languages, so all of them build exactly the same graphs. The benchmark prints the weight of the last path, and the measurement script aborts if the languages disagree; they agreed in every scenario.

What is measured. Each scenario ran in a fresh process 5 times per language (once where a table says so); we report the median over the processes. Inside a process, the query is issued 22 times. The *first* query includes one-time work (adjacency rows, JIT compilation). The *second* is what a short benchmark would report. The *steady-state* time is the median of the 20 further identical queries; it is the time of the shortest-path tree plus the first path, and is what the tables call "tree". Separating the three matters for JIT-compiled runtimes [20, 4] and changes

Table 3: Eppstein, corner to corner on one-way grids (ms). Tree: steady-state query; next: the remaining $k-1$ paths, written out as edge lists. Fastest per group in bold (within 3%).

	tree (steady state)			next paths		
	C#	Go	Rust	C#	Go	Rust
200^2, $k = 10^4$	6.5	7.7	**5.8**	79.4	40.5	**30.8**
500^2, $k = 10^4$	**54.8**	67.0	**56.3**	196	108	**88.3**
1000^2, $k = 10^4$	260	316	273	394	349	277
1000^2, $k = 10^5$	260	314	277	3,094	3,188	2,804

Table 4: Enumerating 10^5 paths on the 10^6-vertex grid after the tree is known: time with and without writing out the paths, and the memory allocated and garbage collections during the enumeration with paths.

	C#	Go	Rust
paths written out (ms)	3,094	3,188	2,804
weights only (ms)	144	44.4	**28.4**
allocated, paths written out (MB)	839	855	837
garbage collections	6	5	–
total GC pause (ms)	119.6	0.5	–
allocated, weights only (MB)	32	37	15

the conclusions (section 6). "Next" is the time for the remaining $k-1$ paths, measured once per process after the last of these queries. The benchmark also records the garbage collections, their total pause time and the bytes allocated during "next" (from the runtimes' own counters; in Rust from a counting allocator), the live heap after the first query, and the script records the peak resident memory of the process. As a measure of dispersion we computed, for every time of at least 1 ms, the range (largest minus smallest value over the 5 processes) relative to the median: its median over all entries is 15%, its 90th percentile 30% and its maximum 81%. Differences between languages below roughly 30% should therefore not be interpreted; we come back to this in section 8.

Eppstein, corner to corner (tables 3 and 4). Grids with $4 \cdot 10^4$ to 10^6 vertices, paths across the whole graph. All three languages return the same path weights. In steady state the tree of the 10^6-vertex grid takes 260 ms (C#), 316 ms (Go) and 273 ms (Rust). On every grid the three are within a factor of 1.2–1.3 of each other, which is inside the dispersion of the measurements, and no language is fastest on all sizes. The next 10^4 paths take 277–394 ms on the largest grid. On the small grid C# needs 79 ms where Rust needs 31 ms; this is again a warm-up effect, because the enumeration code runs here for the first time (section 6). For 10^5 paths all three need 28–32 µs per path.

Table 4 shows where this time goes. Asking only for the weights of the 10^5 paths takes 28.4–144 ms, or 0.3–1.4 µs per path: 95–99% of the enumeration time is spent writing out paths of about 2000 edges each, as the $O(k\ell)$ term of table 1 predicts, and all three languages allocate about the same 837–855 MB for them. The enumeration proper, the part that Eppstein's algorithm is about, is two orders of magnitude cheaper than the output and cheaper than the tree. The weights-only iterator of C# is several times slower than the other two; we did not investigate why.

Detour bound (table 5). Two vertices in the middle of the 10^6-vertex grid, 50 columns and 50 rows apart. Without a bound, the tree covers every vertex that can reach the goal; with paths limited to 10% above the shortest one, it covers only a neighborhood of the goal. Without a

Table 5: Nearby vertices in the middle of the 10^6-vertex grid, $k = 10^4$ (ms). First: first query of the process (includes one-time work such as JIT compilation and building adjacency rows); tree: steady state.

	first query			tree (steady state)			next paths		
	C#	Go	Rust	C#	Go	Rust	C#	Go	Rust
no corridor	266	114	**95.6**	**69.9**	86.5	75.0	29.4	11.7	**11.2**
`MaxDetourRatio` = 0.1	60.7	**26.7**	31.8	2.3	0.58	**0.52**	29.4	**10.8**	**11.0**

Table 6: Building the 10^6-vertex grid ($2.1 \cdot 10^6$ edges), in ms; live memory of the graph with one query and peak resident memory of the whole benchmark process (built by index, $k = 10^4$).

	C#	Go	Rust
by index	460	661	279
by label strings	1,326	1,175	863
DIMACS text import	704	1,020	473
live memory of the graph (MB)	84	94	112
peak resident memory of the process (MB)	406	344	220

bound the steady-state tree takes 70–86 ms. With the bound it takes 0.52–2.3 ms, a factor of 31–148 less depending on the language, and 10^4 paths take 10.8–29.4 ms. In absolute terms the bound matters far more than any language or preprocessing choice. This is the favorable case for the bound: by section 2.4 it searches a ball around the goal, which is small here because start and goal are close. The C# value of 2.3 ms is 4.4× the others and is not a steady state: a query this short does not run long enough in 22 calls to be recompiled with optimizations in every process (the fastest process reached 0.57 ms).

Simple paths (table 7). Two-way grids, where simple paths matter. Each simple path costs many A* spur searches, so the times are much higher than for walks: 687–964 ms for 99 further paths and 5.4–7.6 s for 999 on the $4 \cdot 10^4$-vertex grid, that is 5–10 ms per path where a walk costs microseconds, in line with the bounds of table 1. C# and Rust are equally fast here; Go is slower by about 40% and allocates 940 MB for $k = 1000$ where C# and Rust allocate 502 and 568 MB. A likely reason is that its trie and its duplicate check use maps keyed by strings built from the edge lists; we did not verify this by profiling. With a detour bound on the 10^6-vertex graph, 100 simple paths take 32–54 ms.

Arc flags and contraction hierarchy (table 8). The two-way grid with $4 \cdot 10^4$ vertices allows a direct comparison. Arc flags need 10.3–12.6 s of preprocessing with 4 regions and 45–53 s with 16 regions (built-in partition), and make the tree 15–21% and 15–25% faster. As expected from section 2.6, the gain is small, at the edge of what the measurement can resolve, and bought with a preprocessing time of more than a thousand queries. The contraction hierarchy is cheaper to build (2.5–3.0 s) and more effective: PHAST computes the tree 2.1–4.0× faster than Dijkstra's algorithm. On the 10^6-vertex grid the contraction takes 66–79 s and creates 3,051,496 shortcuts, about 1.5 per original edge; PHAST then needs 82–107 ms instead of 260–316 ms (2.4–3.6×), so the preprocessing is amortized after 310–520 tree computations. Go and Rust produce exactly the same number of shortcuts, because their priority queues break ties identically; C# differs by less than 0.1%. The speed-up is well below the factor of more than ten that PHAST achieves on road networks [10]. There are three reasons: grids have no hierarchy, so m^+ is large; our sweep is single-threaded and does not use SIMD; and our times include rebuilding the tree from the original edges. Whether the technique pays off for k shortest paths on road networks is therefore

Table 7: Simple paths (Yen–Lawler with A*) on two-way grids: time for the paths after the first (ms).

	C#	Go	Rust
200^2, $k = 100$	687	964	710
200^2, $k = 1000$	5,450	7,612	5,441
1000^2 local, ratio 0.1, $k = 100$	53.8	50.7	**32.3**

Table 8: Preprocessing techniques: preprocessing time and the steady-state shortest-path tree, in ms, and the number of shortcuts (Rust; C# differs slightly because of tie-breaking). The 16-region arc flags and the 1000^2 contraction were run in one process per language.

	preprocessing			tree (steady state)			
	C#	Go	Rust	C#	Go	Rust	shortcuts
200^2 two-way: Dijkstra	–	–	–	**7.9**	10.5	9.3	–
arc flags, 4 regions	11.2 s	12.6 s	10.3 **s**	**6.8**	8.6	7.4	–
arc flags, 16 regions	45.3 **s**	52.7 s	45.3 **s**	**6.7**	7.9	**6.9**	–
CH + PHAST	3,008	2,793	2,501	3.7	3.1	**2.3**	179,980
1000^2 one-way: Dijkstra	–	–	–	260	316	273	–
CH + PHAST	79.2 s	70.9 s	65.8 **s**	107	87.5	**81.5**	3,051,496

still open (section 8).

The 2010 original (fig. 3). On acyclic grids, where it terminates at all, the original implementation's first query enumerates every path: 156 ms for 10×10 (48,620 paths), 1.8 s for 11×11 and 40 s and 132 MB for 12×12 (705,432 paths). The current version needs about 0.1 ms for the first path and less than 3 ms for the next 999, independent of the number of paths in the graph. On graphs with cycles, the original does not terminate. This comparison shows the difference between an eager prototype and a lazy algorithm, not a tuning effect; it was measured once, in one process.

6 Discussion: garbage collection and language

Table 9 repeats three scenarios with different runtime settings.

JIT warm-up is the largest language effect, and a measurement trap. With default settings, .NET first compiles methods quickly without optimization and recompiles them with optimizations after they have been called often or have looped long enough. The second query of a process therefore runs partly unoptimized code: on the 200^2 grid it takes 28.8 ms in C#, against 5.3 ms in steady state and 7.6 ms when tiered compilation is disabled. Across all scenarios the second query of C# is up to 5.0× slower than its steady state, while Go and Rust, which are compiled ahead of time, show no such difference. Comparing the languages on the second query gives gaps of up to 5.2×; in steady state the largest gap is 1.6× over all queries that take at least 1 ms. The first version of this paper reported the second query as "the tree" and concluded that Rust was the fastest in most scenarios; with steady-state times C# is the fastest or tied on most trees computed with Dijkstra's algorithm. Both views are legitimate, for different users: a command-line tool that answers one query lives in the first regime, a service in the second. The same effect appears in the time for the next paths on short runs (C# needs 86 ms with and 48 ms without tiered compilation on the small grid) and in the sub-millisecond query of table 5.

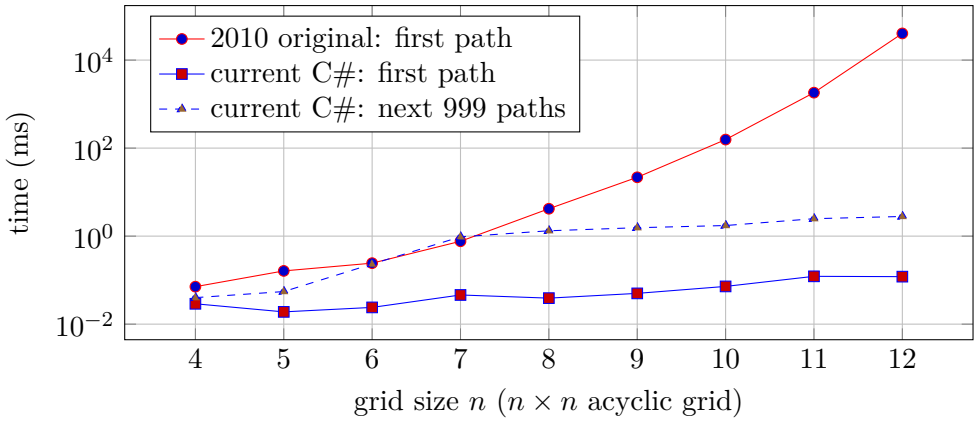


Figure 3: The 2010 implementation enumerates all $\binom{2(n-1)}{n-1}$ paths in the first query; the current one computes one tree and then each path on demand.

Table 9: Runtime settings (median ms). 2nd: second query of the process; steady: median of 20 further queries; next: the remaining paths; GCs: garbage collections while the next paths are enumerated.

	Eppstein 200^2, $k = 10^4$				Eppstein 1000^2, $k = 10^5$				Yen 200^2, $k = 10^3$			
	2nd	steady	next	GCs	2nd	steady	next	GCs	2nd	steady	next	GCs
C# default	28.8	5.3	85.7	0	279	270	3,339	6	39.9	8.9	6,003	4
C# gen0 budget 256 MB	28.1	7.5	91.5	0	268	271	3,219	3	37.8	7.7	5,492	1
C# non-concurrent GC	27.7	7.1	87.3	0	255	262	3,186	6	37.9	8.1	5,418	3
C# without tiered JIT	7.6	6.4	48.2	0	275	262	3,063	6	9.6	10.3	5,626	4
Go default	11.0	8.8	46.7	3	318	320	3,228	5	13.9	11.0	7,882	10
Go `GOGC=off`	9.1	8.2	43.7	0	340	320	3,494	0	12.3	11.5	8,163	0
Rust	6.6	6.8	36.1	–	302	295	2,990	–	10.1	9.3	6,222	–

Garbage collection costs a few percent. Enumerating 10^5 paths allocates about 840 MB in all three languages (table 4), almost all of it for the returned paths. Rust frees each path when the caller drops it. .NET runs 6 collections that pause the program for 120 ms in total, 4% of the enumeration time; for Yen's algorithm the share is also 4%. Go runs 5 collections with 0.46 ms of pauses; its collector works concurrently on other cores, so the pause understates its CPU cost but not its effect on the elapsed time of a single-threaded program on a four-core machine. The collections are cheap because the garbage is short-lived and the live data (the graph and the heaps of the query) is small and changes little; the cost of a tracing collector grows with the live data, not with the garbage. Two interventions confirm this. Turning Go's collector off (`GOGC=off`) does not make the enumeration faster (3,494 against 3,228 ms) and raises the peak resident memory from 390 to 1,041 MB; the memory that is no longer reused must be obtained fresh from the operating system, which costs as much as collecting it did. A 256 MB budget for the youngest generation halves the number of .NET collections without changing the time. As an upper bound for everything that distinguishes the languages in this workload, including memory management, Rust enumerates 9–12% faster than the other two, which is below the dispersion of our measurements.

What the collectors cost is memory. The live heap of the 10^6-vertex graph with one query is similar, 84 MB (C#), 94 MB (Go) and 112 MB (Rust; C# and Go share the caller's label strings, which are therefore not counted). The peak resident memory of the whole process is 406, 344 and 220 MB (table 6): the garbage-collected runtimes need 1.6–1.8× the memory of the Rust process for the same work. This is the trade-off described by Hertz and Berger [24]: collectors

match explicit memory management in speed when they are given more memory.

What remains is compilers and libraries. Apart from warm-up, the differences are small and without a clear order. Rust builds the graph fastest (279 ms by index against 460 and 661 ms). In the search loops of Dijkstra's algorithm and A*, all three do the same array accesses with bounds checks; Go is the slowest there by 20–30%, which is about the size of the dispersion. Library choices matter at this level: the 4-ary heap of .NET's `PriorityQueue` against our binary heaps in Go and Rust, and the hash tables behind the trie and the duplicate check of Yen's algorithm. These are properties of the three implementations, not of the languages (section 8).

Engineering effort. The Rust port was the third implementation and was generated by Claude from the Go version. Two Rust-specific properties stood out. The borrow checker turned the "graph changed during iteration" error of C# and Go into a compile-time error, at the price of a slightly less convenient iterator. And the most natural Rust structure for persistent heaps, `Rc` pointers, was replaced by index-based arenas, which are both faster and simpler to reason about than reference counting.

7 AI-assisted development

7.1 Process

The whole project is an experiment with Claude [3]. The author worked with it as a coding agent in a cloud development environment with access to the repository. Claude read and wrote the code, ran builds, tests, benchmarks and a fuzzer, searched the literature, opened pull requests and reacted to CI results; the author decided what to do next, chose among the options it proposed, and merged. All work happened in three sessions over three days (8–10 November 2026); 11 pull requests were merged before the first version of this paper. The author's requests were short and informal, usually one sentence, and never a specification.

7.2 What the human contributed

Three kinds of input shaped the result. *The old algorithm* fixed the interface (labels, groups, `FindShortestPath`/`FindNextShortestPath`), which goes back to ideas from a Delphi implementation [21], and the domain; much of the later work kept that interface or broke it only when the author allowed it. *Open questions* (what would improve the project most, how to make it much faster, what is missing for a high-quality library) delegated the search for improvements to Claude, while the author kept the decisions by choosing among the proposed items. *Expert ideas* (arc flags with a real partitioner, contraction hierarchies, the idea for a paper) came from a colleague's review and redirected the work towards algorithm engineering. Explicit constraints, such as excluding multithreading or allowing breaking interface changes, were the most effective requests.

7.3 What Claude did, and where it failed

Claude wrote all code, tests, documentation and benchmarks of version 2, ran every measurement in this paper, and proposed most of the ideas that were implemented. It also argued *against* ideas when its measurements did not support them: it first advised against contraction hierarchies, because the measured bottleneck of long queries was the number of spur searches, and after being asked to implement them it reported that the gain on grids (2.4–3.6× for the tree) is below the order of magnitude reported for road networks. The failures in the code were caught by tests, not by review:

- Hand-computed expected values in new tests were wrong several times (e. g. the weights of simple paths in a small two-way graph, and the edges of Eppstein's example typed from memory in the Rust tests); in each case brute force or the shared golden data showed that the library was right and the expectation wrong.
- Fuzzing found that the Go DIMACS importer treated some Unicode white space differently from line splitting, and that Claude's own test harness had an $O(n^2)$ sort.
- A scripted text edit broke a Go struct tag; the build caught it.
- The first Rust port stored labels twice; the cross-language memory comparison exposed it.
- The environment could not download the DIMACS road networks (the server is not reachable from it), so all experiments use synthetic grids (section 8).

7.4 A critical review of the first version of this paper

The text has no test suite. On the third day the author asked Claude to review the first version of this paper critically, as a referee would. The review found errors of a different kind than those in the code:

- *A wrong citation.* The lazy variant of Eppstein's algorithm was attributed to Jiménez and Marzal's 1999 paper, which introduces the recursive enumeration algorithm; the lazy variant is their 2003 paper. Relevant related work (K*, near-shortest paths, Feng, Hershberger et al.) was missing, which made the detour bound and the A* spur searches look more original than they are.
- *Claims without data.* The text stated a 20% gain for arc flags, a "3–10×" gain for PHAST and that writing out paths dominates the enumeration; none of these had a measurement in the paper. Measured, the first and the third hold (tables 4 and 8); the second does not: the gain is 2.1–4.0× and 2.4–3.6×.
- *A confounded headline result.* The main table reported the second query of each process as "the tree" and showed C# 3–4× slower than Rust on a small grid. The text explained this as JIT warm-up, but the table and the abstract's "Rust is the fastest in most scenarios" still rested on it. With steady-state measurements the ranking changes (section 6).
- *An indirect argument.* "Garbage collection is not a factor" was inferred from wall-clock times with the collector disabled, without checking how often it ran or how long it paused, and although disabling a collector changes more than the collections.
- *Small inaccuracies.* The number of paths within a detour bound was called finite without excluding zero-weight cycles, "allocation-free" described a layout whose enumeration allocates, and a stated maximum gap of 4× contradicted a 6× entry in a table of the same paper.

All of these read plausibly, and none would have been caught by the tests. They share a pattern with the wrong expected values above: statements produced from memory or from an explanation that sounded right, where a measurement or a look at the source was possible. The remedy was the same as for the code. The benchmark programs now measure what the text claims, and every measured number in the running text is inserted from the data files by a macro; the statements about the 2010 baseline and about cited work are the exceptions.

7.5 Observations

The decisive factor for reliability was the test infrastructure that Claude built early: golden data shared by all implementations, brute-force oracles and fuzzing. It made large refactorings (object graph to arrays; three languages) cheap to verify and let the author accept changes without reading every line. Claude was most useful as an algorithm engineer when asked open questions and given freedom, and most precise when given constraints. It never needed a formal specification; it needed a way to check itself. Where no such check existed, in the prose of a paper, it produced confident errors, and found them only when asked explicitly to doubt its own text.

8 Threats to validity

Inputs. All experiments use synthetic grids with uniform random weights. They have no hierarchy, which makes them a hard case for contraction hierarchies, and the one-way grids are almost acyclic. Results on road networks (DIMACS [12], OpenStreetMap) may differ in both directions; this is the most important missing experiment. Each scenario uses one graph (one seed) and one s–t pair, so the times do not average over queries.

Measurement. The machine is a shared cloud VM with unknown neighbors; we report medians of fresh processes and the dispersion, but no confidence intervals. The contraction of the largest graph and the 2010 baseline were measured once. The live-heap numbers are measured differently in each language (after a full collection in C# and Go, allocator bytes in Rust), and peak resident memory includes the benchmark's own edge list.

Language comparison. There is one implementation per language, all generated by the same AI model, the Go and Rust versions as ports. The implementations are not identical in every data structure (objects vs. arenas for the enumeration, a 4-ary vs. binary heaps, different hash tables), so differences between languages cannot be attributed to the garbage collector or the compiler alone, and other programmers would obtain other ratios [34]. The conclusions hold for single-threaded code with a data layout that avoids allocation in the search loops; they say nothing about allocation-heavy or concurrent programs.

Case study. The case study has a sample size of one author and one AI model. This paper itself, including the review in section 7, was generated with Claude, so its account of the process is not independent; the repository history and the data files are the primary evidence.

9 Conclusion

A small 2010 prototype became a library that combines Eppstein's enumeration, a detour bound, Yen–Lawler with exact A*, arc flags and PHAST on a contraction hierarchy, in three languages with identical results. On the grids we measured, algorithmic choices changed the running time by orders of magnitude, while the three languages stayed within a factor of 1.6 of each other in steady state once the data layout avoided allocation in the search loops; warm-up of the JIT-compiled runtime was a larger effect than garbage collection, whose cost showed in memory rather than in time. Promising next steps are experiments on real road networks, a bidirectional or K*-style construction that replaces the ball of the detour bound by the exact region, and the sidetrack-based simple-path algorithms [29, 37] combined with (R)PHAST or CCH for the repeated tree computations. The code, the data and the scripts that produced every measurement in this paper are available in the repository [15].

Acknowledgments

The author thanks the following people for verbal discussions: Dr. Henrik Reinstädtler [35], and Jürgen Güntherodt [21], who also contributed ideas for the interface, based on a Delphi implementation.

References

[1] Ittai Abraham, Daniel Delling, Andrew V. Goldberg, and Renato F. Werneck. Alternative routes in road networks. *ACM Journal of Experimental Algorithmics*, 18, 2013.

[2] Husain Aljazzar and Stefan Leue. K*: A heuristic search algorithm for finding the k shortest paths. *Artificial Intelligence*, 175(18):2129–2154, 2011.

[3] Anthropic. Claude code: an agentic coding tool. https://claude.com/claude-code. Accessed November 2026.

[4] Edd Barrett, Carl Friedrich Bolz-Tereick, Rebecca Killick, Sarah Mount, and Laurence Tratt. Virtual machine warmup blows hot and cold. *Proceedings of the ACM on Programming Languages*, 1(OOPSLA):52:1–52:27, 2017.

[5] Hannah Bast, Daniel Delling, Andrew Goldberg, Matthias Müller-Hannemann, Thomas Pajor, Peter Sanders, Dorothea Wagner, and Renato F. Werneck. Route planning in transportation networks. In *Algorithm Engineering: Selected Results and Surveys*, volume 9220 of *LNCS*, pages 19–80. Springer, 2016.

[6] Thomas H. Byers and Michael S. Waterman. Determining all optimal and near-optimal solutions when solving shortest path problems by dynamic programming. *Operations Research*, 32(6):1381–1384, 1984.

[7] W. Matthew Carlyle and R. Kevin Wood. Near-shortest and K-shortest simple paths. *Networks*, 46(2):98–109, 2005.

[8] Theodoros Chondrogiannis, Panagiotis Bouros, Johann Gamper, Ulf Leser, and David B. Blumenthal. Finding k-shortest paths with limited overlap. *The VLDB Journal*, 29:1023–1047, 2020.

[9] Clark A. Crane. Linear lists and priority queues as balanced binary trees. Technical Report STAN-CS-72-259, Stanford University, Computer Science Department, 1972.

[10] Daniel Delling, Andrew V. Goldberg, Andreas Nowatzyk, and Renato F. Werneck. PHAST: Hardware-accelerated shortest path trees. *Journal of Parallel and Distributed Computing*, 73(7):940–952, 2013.

[11] Daniel Delling, Andrew V. Goldberg, and Renato F. Werneck. Faster batched shortest paths in road networks. In *11th Workshop on Algorithmic Approaches for Transportation Modelling, Optimization, and Systems (ATMOS 2011)*, volume 20 of *OASIcs*, pages 52–63, 2011.

[12] Camil Demetrescu, Andrew V. Goldberg, and David S. Johnson, editors. *The Shortest Path Problem: Ninth DIMACS Implementation Challenge*, volume 74 of *DIMACS Series in Discrete Mathematics and Theoretical Computer Science*. American Mathematical Society, 2009.

[13] Julian Dibbelt, Ben Strasser, and Dorothea Wagner. Customizable contraction hierarchies. *ACM Journal of Experimental Algorithmics*, 21, 2016.

[14] Edsger W. Dijkstra. A note on two problems in connexion with graphs. *Numerische Mathematik*, 1:269–271, 1959.

[15] Nick Dinges. k-shortest-path: k shortest paths in C#, Go and Rust. https://github.com/NickDinges/k-shortest-path, 2026.

[16] James R. Driscoll, Neil Sarnak, Daniel D. Sleator, and Robert E. Tarjan. Making data structures persistent. *Journal of Computer and System Sciences*, 38(1):86–124, 1989.

[17] David Eppstein. Finding the k shortest paths. *SIAM Journal on Computing*, 28(2):652–673, 1998.

[18] Gang Feng. Finding k shortest simple paths in directed graphs: A node classification algorithm. *Networks*, 64(1):6–17, 2014.

[19] Robert Geisberger, Peter Sanders, Dominik Schultes, and Daniel Delling. Contraction hierarchies: Faster and simpler hierarchical routing in road networks. In *Experimental Algorithms (WEA 2008)*, volume 5038 of *LNCS*, pages 319–333. Springer, 2008.

[20] Andy Georges, Dries Buytaert, and Lieven Eeckhout. Statistically rigorous Java performance evaluation. In *Proceedings of OOPSLA 2007*, pages 57–76. ACM, 2007.

[21] Jürgen Güntherodt. Verbal discussions and ideas for the interface, based on a Delphi implementation. Personal communication. https://www.linkedin.com/in/juergen-guentherodt/, 2026.

[22] Peter E. Hart, Nils J. Nilsson, and Bertram Raphael. A formal basis for the heuristic determination of minimum cost paths. *IEEE Transactions on Systems Science and Cybernetics*, 4(2):100–107, 1968.

[23] John Hershberger, Matthew Maxel, and Subhash Suri. Finding the k shortest simple paths: A new algorithm and its implementation. *ACM Transactions on Algorithms*, 3(4):45:1–45:19, 2007.

[24] Matthew Hertz and Emery D. Berger. Quantifying the performance of garbage collection vs. explicit memory management. In *Proceedings of OOPSLA 2005*, pages 313–326. ACM, 2005.

[25] Robert Hundt. Loop recognition in C++/Java/Go/Scala. In *Proceedings of Scala Days 2011*, 2011.

[26] Víctor M. Jiménez and Andrés Marzal. Computing the K shortest paths: A new algorithm and an experimental comparison. In *Algorithm Engineering (WAE 1999)*, volume 1668 of *LNCS*, pages 15–29. Springer, 1999.

[27] Víctor M. Jiménez and Andrés Marzal. A lazy version of Eppstein's K shortest paths algorithm. In *Experimental and Efficient Algorithms (WEA 2003)*, volume 2647 of *LNCS*, pages 179–191. Springer, 2003.

[28] Guy L. Steele Jr., Doug Lea, and Christine H. Flood. Fast splittable pseudorandom number generators. In *Proceedings of OOPSLA 2014*, pages 453–472. ACM, 2014.

[29] Denis Kurz and Petra Mutzel. A sidetrack-based algorithm for finding the k shortest simple paths in a directed graph. In *27th International Symposium on Algorithms and Computation (ISAAC 2016)*, volume 64 of *LIPIcs*, pages 49:1–49:13, 2016.

[30] Ulrich Lauther. An extremely fast, exact algorithm for finding shortest paths in static networks with geographical background. In *Geoinformation und Mobilität*, volume 22 of *IfGIprints*, pages 219–230. 2004.

[31] Eugene L. Lawler. A procedure for computing the K best solutions to discrete optimization problems and its application to the shortest path problem. *Management Science*, 18(7):401–405, 1972.

[32] Rolf H. Möhring, Heiko Schilling, Birk Schütz, Dorothea Wagner, and Thomas Willhalm. Partitioning graphs to speedup Dijkstra's algorithm. *ACM Journal of Experimental Algorithmics*, 11, 2006.

[33] Mt-KaHyPar developers. Mt-KaHyPar: Multi-threaded Karlsruhe graph and hypergraph partitioner. https://github.com/kahypar/mt-kahypar. Accessed November 2026.

[34] Lutz Prechelt. An empirical comparison of seven programming languages. *IEEE Computer*, 33(10):23–29, 2000.

[35] Henrik Reinstädtler. Verbal discussions. Personal communication. https://www.linkedin.com/in/henrixapp/, 2026.

[36] Jin Y. Yen. Finding the k shortest loopless paths in a network. *Management Science*, 17(11):712–716, 1971.

[37] Ali Al Zoobi, David Coudert, and Nicolas Nisse. Finding the k shortest simple paths: Time and space trade-offs. *ACM Journal of Experimental Algorithmics*, 28, 2023.